

Offloading ECG Inference from ESP32 to Host: A Real-Time Layered BLE Transmission Study

Ashkan Zahabiun (#200464070)

Department of Computer Science
North Carolina State University
Raleigh, USA
azahabi2@ncsu.edu

Tyler Klick

Department of Computer Science
North Carolina State University
Raleigh, USA
tjclick@ncsu.edu

Zhishan Guo

Associate Professor
Department of Computer Science
North Carolina State University
Raleigh, USA
zguo32@ncsu.edu

Abstract—Continuous cardiac monitoring on resource-constrained embedded hardware demands a careful balance between real-time inference, wireless throughput, and connectivity resilience. This independent study extends a prior PhD-level testbed [2] that ran a TinyML ECG classifier directly on an ESP32 microcontroller, consuming 40 KB of SRAM for the TFLite tensor arena and coupling inference latency to the BLE transmission schedule. We redesign the system so the ESP32 is a pure streaming node: a 360 Hz hardware timer ISR collects ECG samples from a LittleFS-resident MIT-BIH recording, splits each 256-sample window into four interleaved BLE layers (588 bytes total, sustaining ≈ 827 bytes/s at under 4% of available BLE bandwidth), and transmits them to a Python host over an application-layer ARQ channel. The host runs TFLite inference, recovering risk scores that the INT8-quantized model produces at an effective detection threshold of 0.45—a value determined empirically by observing the model’s discrete output steps ($\Delta = 1/256 \approx 0.004$) against MIT-BIH arrhythmia records. A novel offline ring buffer (30 windows, portMUX_TYPE-protected) accumulates ECG data during BLE disconnections; on reconnect, buffered windows are delivered with priority before live streaming resumes, ensuring zero data loss during short outages. Hardware validation on a DOIT ESP32 DEVKIT V1 confirms end-to-end operation: normal windows score $\text{risk} < 0.1$, while arrhythmic windows from record 208 (bigeminy) score 0.45–0.48, triggering anomaly alerts. The hardware-in-the-loop for scheduling tests, the calibrated Proposed scheduler preserves 98.69% of R-peaks at 30% packet loss and achieves the lowest reconstruction MSE at 60–75% packet loss.

I. INTRODUCTION

Cardiovascular disease is the leading cause of death globally [1], and continuous ambulatory ECG monitoring is a proven intervention for early arrhythmia detection. Wearable sensors built around low-power microcontrollers such as the ESP32 are compelling for this role, but deploying a machine learning classifier on a device with 520 KB of SRAM and no floating-point unit imposes real constraints on model size, inference latency, and firmware complexity.

A. Prior Work

The system extended in this independent study is described in [2]. That system ran a complete on-device inference

pipeline: a 360 Hz hardware timer ISR collected ECG windows; a 4-bit error-diffusion quantizer compressed 11-bit ADC samples to reduce the dynamic range before model input; the on-device classifier produced a float32 risk score per window; and that score fed a utility-aware BLE scheduler that split each 256-sample window into one mandatory BASE layer and three optional ENHANCEMENT layers. The BASE layer carries every 4th sample (indices 0, 4, 8, ...) and is sufficient to reconstruct the coarse rhythm and R-peak positions; the ENHANCEMENT layers supply the interleaved samples that recover full waveform morphology. The on-device model consumed ≈ 40 KB of SRAM for its tensor arena and added 5–15 ms of inference latency per window.

The scheduler introduced in [2] is a utility-aware priority function:

$$U = \frac{\alpha}{d' - t} + \beta r - \gamma D \quad (1)$$

where d' is a risk-adjusted deadline, $r \in [0, 1]$ is the segment risk score, D is a per-layer distortion weight (higher for BASE), and t is the current time. Coefficients $\alpha = 2500$, $\beta = 10$, $\gamma = 11$ were set so that BASE layers always dominate ENHANCEMENT layers on urgency alone, and high-risk segments dominate low-risk ones. In the refreshed hardware-in-the-loop evaluation, the calibrated scheduler preserves 98.69% of R-peaks at 30% packet drop and gives the lowest MSE at 60–75% drop, showing that utility-aware ordering preserves the structural ECG backbone under significant channel loss. This advantage depended entirely on having accurate real-time risk scores; the scheduler degrades to worse-than-EDF when risk information is absent.

B. This Independent Study

This work asks: what changes when inference moves from the sensor to the receiving device, and how should the sensor behave when the BLE link is interrupted? Moving inference to a capable host allows the use of a full-precision model that is not constrained by SRAM or INT8 quantization, and model updates no longer require firmware reflash. Critically, the on-device classifier in [2] served as a real-time proxy for signal quality - providing risk scores that drove scheduling decisions. Offloading replaces this with a *retrospective* proxy: risk scores arrive after the segment has been transmitted, so they inform

scheduling of *future* segments rather than the current one. The feedback channel (Section VII) closes this loop.

The second question - sensor resilience during connectivity loss - has no counterpart in [2]. When the BLE link drops, the sensor must continue collecting and preserve data; on reconnect, the host can classify all buffered windows including any arrhythmias that occurred during the outage.

Contributions:

- 1) A redesigned ESP32 firmware that removes on-device inference, streams raw ECG at 360 Hz, and maintains a 30-window offline ring buffer that preserves data during BLE disconnections for priority delivery on reconnect - enabling the host to classify segments collected during outages.
- 2) A Python inference host that runs the full-precision TFLite classifier on *all* received windows - live and offline alike - providing retrospective anomaly detection on buffered data, with a detection threshold of 0.45 calibrated empirically to the INT8 output quantization boundary.
- 3) A quantitative BLE bandwidth analysis showing ≈ 855 bytes/s sustained throughput - under 4% of available bandwidth - with headroom for risk-score feedback.
- 4) Hardware validation including anomaly detection on MIT-BIH record 208 (bigeminy) and two complete offline-buffer-fill-and-flush cycles.

II. BACKGROUND

A. ECG and Arrhythmia Detection

An ECG records the heart's repeating PQRST complex; arrhythmias such as PVCs, atrial fibrillation, and ventricular flutter manifest as morphological deviations. The MIT-BIH Arrhythmia Database [3] provides 48 half-hour two-lead recordings at 360 Hz with beat-by-beat annotation. It is the standard benchmark for automated classification [9], [10] and the data source for this project. Records vary widely in arrhythmia density: record 100 is predominantly normal sinus rhythm; record 208 exhibits sustained bigeminy throughout.

B. TinyML, Offloading, and INT8 Quantization

TinyML deploys ML models on microcontrollers [5] using TensorFlow Lite for Microcontrollers (TFLM) with INT8 post-training quantization [6], [15]: $q = \text{round}(x/s) + z$, where per-tensor scale s and zero-point z are calibrated from a representative dataset. Quantization reduces model memory by $4\times$ versus float32 and enables execution on hardware without a floating-point unit, typically at a small accuracy cost. Inference offloading [7] moves model execution from the constrained sensor to a capable host device. The key trade-off is that risk scores are no longer available before the first BLE packet is sent; instead they are computed after the segment is fully received, introducing a one-window feedback delay. For continuous ECG monitoring this is acceptable: arrhythmia episodes span multiple windows, so the prior window's score can serve as a risk proxy for the current one. A further benefit

is *model agility*: the host can receive model updates without reflashing the sensor firmware.

C. Layered ECG Transmission and Scheduling

The transmission scheme in [2] encodes each 256-sample ECG window into four interleaved 64-sample layers. The mandatory BASE layer (layer 0) carries every 4th sample and is sufficient for rhythm detection and R-peak localization; the three ENHANCEMENT layers supply the remaining interleaved samples to recover full morphology. A receiver that obtains only the BASE layer can reconstruct a coarse but clinically useful signal; each additional ENHANCEMENT layer refines the reconstruction. Layer l carries samples at indices $l, l+4, \dots, l+252$; reconstruction reverses the stride: $\text{win}[l+4k] = \text{layer}_l[k]$.

Three schedulers govern which packets are sent first under channel congestion: **EDF** [8] (Earliest Deadline First), **DA-EDF** (Distortion-Aware EDF, which applies a tighter effective deadline to BASE layers), and the **Proposed** utility function from Eq. 1 that incorporates risk scores directly.

D. BLE ARQ and Bandwidth

BLE GATT communication uses two characteristics: TX (NOTIFY, ESP32 $\rightarrow$ Python) and ACK (WRITE_NR, Python $\rightarrow$ ESP32). Each 147-byte notification carries a 19-byte header and 128 bytes of int16 samples; an ACK is 5 bytes. Per window: $4 \times 147 + 4 \times 5 = 608$ bytes bidirectional at one window per 711 ms:

$$\text{Throughput} \approx \frac{608}{0.711} \approx 855 \text{ bytes/s (6.8 kbps)} \quad (2)$$

The BLE 4.2 PHY [11] delivers $\approx 150\text{--}250$ kbps effective throughput; our protocol uses 3–5% (Table I). This low utilization means the link is unlikely to be the bottleneck even in RF-congested clinical environments, and leaves substantial headroom: the risk-score feedback extension (Section VII) adds under 0.1% overhead, and a second sensor channel could be added without exhausting BLE capacity.

TABLE I
BLE BANDWIDTH BUDGET (PER 256-SAMPLE WINDOW)

Component	Bytes	Share
ECG payload ($64 \times 2 \times 4$ layers)	512	87.1%
Header overhead (19×4)	76	12.9%
ACK upstream (5×4)	20	-
Bidirectional per window	608	-
Sustained rate	855 bytes/s, $\approx 3\text{--}5\%$ BLE capacity	-

III. SYSTEM DESIGN

A. Architecture Overview

Table II contrasts the original on-device architecture with the offloaded design. The ESP32 becomes a pure streaming node; all inference moves to Python. The offline ring buffer is a new architectural layer with no counterpart in [2].

Key offline behavior: when the BLE link drops, the ESP32 continues sampling at 360 Hz and stores each completed 256-sample window in a circular ring buffer (up to 30 windows, ≈ 21 seconds of ECG). The sensor holds this data persistently until a client reconnects. On reconnect, the buffered windows are transmitted to the host *first* - ahead of any new live data - tagged with the offline source flag so the host can distinguish them. Only after all buffered windows have been delivered does the system resume normal live streaming. The Python host runs the TFLite classifier on every buffered window as it arrives, so arrhythmias that occurred *during* the outage are detected and flagged retrospectively.

B. BLE Wire Format

Each notification is 147 bytes (little-endian): layer (uint8), seg_id (uint32), arrival_ms (uint32), deadline_ms (uint32), risk (float32), n_samples (uint16), samples (int16 $\times$ 64). ACKs are 5 bytes: uint32 seg_id + uint8 layer. The risk field in the BLE packet serves as a *source flag only*: **0.0**=live data; **-1.0**=offline-buffered. This flag tells the host where the data came from; the host independently computes its own risk score via TFLite inference on the raw samples, regardless of source. The **-1.0** value is chosen because it is outside the model's valid output range $[0, 1]$ and is never produced by a real inference call.

C. Online vs. Offline Data Paths

The system maintains two distinct data paths depending on BLE connectivity. Table III summarizes the state machine governing which path is active.

Signal quality. The 360 Hz hardware timer ISR runs continuously regardless of BLE state - samples are never dropped at the ADC level. Signal quality (amplitude, morphology, timing) is identical in both paths because the same ADC pipeline produces both online and offline windows. What differs is the *transmission timing*: online windows are delivered within ≈ 711 ms of collection; offline windows may be delayed by seconds to tens of seconds.

Flush ordering and partial-flush coherence. Buffered windows are enqueued and flushed in FIFO order. The primary reason is not temporal reconstruction - the host already uses segment IDs to handle out-of-order arrivals - but rather *partial-flush coherence*: if the link drops again mid-flush, the remaining windows in the buffer are still the same contiguous set from the original outage. Without FIFO enforcement, a partial flush followed by a re-disconnect would interleave old buffered windows with newly buffered ones in the queue, making it difficult for the host ML pipeline to determine which windows were clinically adjacent. A more principled approach, left as future work, would treat buffered windows as scheduler tasks with their original deadlines, allowing the scheduler to interleave offline and live windows by urgency rather than imposing a strict two-phase flush.

If the ring buffer overflows (more than 30 windows, > 21 s of disconnection), the oldest windows are evicted and a segment ID gap will appear at the host between the last

OFFLINE-flagged and first live segment; clinical reporting should flag any such gap.

Scheduling state across connectivity loss. When the BLE link drops, in-flight packets awaiting ACK are abandoned - their data is not preserved in the offline buffer, which only captures complete windows from the ADC pipeline. On the transmitting side, the scheduler's queued tasks expire during the outage. On reconnect, the scheduler resumes with fresh live windows; the offline buffer contents bypass the scheduler entirely and are sent directly. This means a high-risk window that was in the scheduler queue at the moment of disconnection may or may not have been fully transmitted before the link dropped; any partially-sent window is not recoverable from the buffer.

Flush ordering and the live-data deadzone. During an offline flush, buffered windows are transmitted before each new live window from the current sampling cycle. Because BLE transmission is ACK-limited (≈ 1.4 windows/s), flushing N buffered windows occupies approximately $0.71N$ seconds of link capacity. During this interval, newly collected live windows are processed one at a time after the flush completes; intermediate windows accumulated by the ISR during a long flush will be overwritten in the double buffer and lost. This creates a *live-data deadzone* proportional to the offline buffer depth: a full 30-window flush produces up to 21 additional seconds of unclassified live data beyond the original disconnection window. A more sophisticated implementation would interleave one buffered window with one live window during flush, limiting the deadzone to a single window per cycle. The current design prioritizes simplicity of the flush path at the cost of this additional gap.

D. Host-Side Inference: Live and Offline

A key architectural property of the offloaded design is that the Python host runs TFLite inference on *every* received segment, regardless of whether it originated from the live stream or from the offline ring buffer. The BLE packet's **-1.0** risk flag is read only to set the `is_offline` label in the log output; the actual classification is always computed from the raw ECG samples. An offline window containing a PVC will produce `risk > 0.45` and trigger `*** ANOMALY ***` with `src=OFFLINE`, providing clinically actionable detection of arrhythmias that occurred *during* a connectivity outage. This is the core advantage of host-side inference: the ESP32 does not need to know the risk of a buffered window to preserve it; the host determines risk retrospectively when the data arrives.

Inference pipeline. The host-side classifier replicates the preprocessing chain from [2] to ensure the model receives input from the same distribution it was trained on:

- 1) **4-bit compression:** each 11-bit ADC sample is quantized to a 4-bit integer using error-diffusion (delta-sigma style) quantization. This matches the on-device compression applied during training and reduces the input dynamic range from $[0, 2047]$ to $[0, 15]$, shaping the signal to the distribution the model was calibrated against.

TABLE II
ARCHITECTURE COMPARISON: ON-DEVICE VS. OFFLOADED INFERENCE

Aspect	Tyler’s Testbed [2]	This Work
Inference location	ESP32 TFLM INT8	Python host TFLite
ESP32 SRAM for model	≈40 KB tensor arena	0 KB
Inference latency	5–15 ms per window	2–5 ms per window (M1)
Risk score timing	Before first BLE packet	After all 4 layers received
Disconnect behavior	Data lost	Buffered (30 windows, ≈21 s)
Model updates	Requires firmware reflash	Replace <code>.tflite</code> on host
Signal preprocessing	4-bit compress on ESP32	4-bit compress in Python

TABLE III
ONLINE/OFFLINE STATE MACHINE

State	gConn	gEverCon.	BLE risk flag	Host ML
Pre-connection	false	false	(discarded)	None
Online	true	true	0.0 (live)	TFLite → ANOMALY?
Offline (buffer)	false	true	−1.0 (src flag)	TFLite → ANOMALY?
<i>On reconnect:</i> flush buffered windows (flag = −1.0) in order; host runs ML on each; then resume online.				

- 2) **Z-score normalization:** $\hat{x}_i = (c_i - \mu_c)/(\sigma_c + \epsilon)$, computed per 256-sample window. This removes amplitude and baseline variation, making the classifier invariant to electrode impedance and patient-to-patient amplitude differences.
- 3) **Input quantization:** the normalized float values are mapped to the model’s INT8 input tensor using the learned affine parameters (s_{in}, z_{in}) baked in at training time, making the input representation consistent with the model weights.
- 4) **Model execution:** the TFLite interpreter runs the 3,347-parameter binary classifier.
- 5) **Output dequantization:** the INT8 output is mapped back to a float32 probability using (s_{out}, z_{out}) , giving a risk score in $[0, 1]$.

The output quantization step size is $1/256 \approx 0.004$, meaning the model cannot produce exactly 0.5. Arrhythmic windows from record 208 score in $[0.45, 0.48]$; the detection threshold is set to 0.45 to capture these windows, which the float32 Keras model would classify as abnormal but the INT8 version represents just below the 0.50 boundary.

IV. IMPLEMENTATION

A. ESP32 Firmware: `630.ino`

The firmware replaces the on-device inference pipeline from [2] with a pure streaming design. The ESP32’s two cores are used for separate concerns: one core manages ADC sampling and LittleFS cache refills, while the other handles BLE transmission. A 360 Hz hardware timer interrupt drives sampling into a double buffer; when 256 samples are complete, the buffers swap and the BLE transmission task is notified. The transmission task applies the state machine from Table III: pre-connection windows are discarded; windows collected during a disconnect are enqueued to the offline buffer; on reconnect, all buffered windows are flushed with the offline source flag before live transmission resumes. Stale acknowledgements

from the previous layer are drained before each new layer send to prevent cross-segment interference. ACK timeout is 500 ms, calibrated to macOS CoreBluetooth write round-trips of 200–400 ms (the original 150 ms caused near-universal timeouts).

Offline ring buffer. The buffer is a fixed-capacity circular array of 30 entries, each storing a 256-sample int16 window and its segment ID. Enqueue and dequeue operations are protected by a hardware-level mutual exclusion spinlock, ensuring $O(1)$ bounded execution time without blocking the ISR or higher-priority tasks. When the buffer is full, enqueue evicts the oldest entry so that the most recent windows survive. A one-time connection flag is set on the first client connection and never cleared; this prevents windows collected during the pre-connection advertising phase from appearing as offline-sourced data on the first BLE connection.

NimBLE v2.x migration. The original testbed used NimBLE v1.x; three callbacks changed signatures (`onConnect`, `onDisconnect`, `onWrite` each gained a `NimBLEConnInfo&` parameter). Using the old signatures with `override` produces hard compile errors that present initially as class-naming conflicts. Version 2.x also requires explicit `advertiseOnDisconnect(true)` for automatic re-advertising.

B. Python Host: `main.py`

The Python host scans for the ESP32 by service UUID rather than device name, as macOS CoreBluetooth caches device names unreliably. Incoming BLE notifications are accumulated per segment ID; the classifier is invoked when all four layers arrive or a 3-second timeout expires. A `simulate-disconnect` key command physically closes the BLE connection (rather than merely withholding ACKs), triggering the disconnect callback on the ESP32 and starting real offline buffering - the only way to properly exercise the buffer path. A separate upload utility downloads any MIT-BIH record directly from PhysioNet, applies the signal normalization pipeline

from [2], and transfers the binary to the ESP32's flash storage over a serial handshake protocol.

C. Problem Solving

Several non-obvious issues were resolved during integration:

Preprocessing mismatch. Initial risk scores plateaued at 0.414 on MIT-BIH record 207 - below the threshold but non-zero, suggesting the model was partially responsive. Comparing Python `main.py` inference with `compare_keras_tflite.py` revealed the missing step: the training pipeline applied `compress_signal(bits=4)` before z-scoring, matching the ESP32's `SignalQuantizer(11, 4)`. Without this step, the model receives z-scores of raw [0, 2047] ADC values rather than the z-scores of [0, 15] 4-bit compressed values it was calibrated on. Adding the compress step pushed peak scores to 0.477 on record 208.

INT8 threshold calibration. With the preprocessing fixed, peak arrhythmic risk was 0.477 - still below the theoretical 0.50 threshold. The cause: INT8 dequantization discretizes outputs in steps of $1/256$; the nearest representable value below 0.5 is 0.496 and the model's arrhythmic outputs landed at ≈ 0.477 . Setting `ANOMALY_THRESHOLD = 0.45` resolved this without retraining.

Simulate-disconnect. Early testing withheld ACKs to simulate a dropout, but the ESP32's `gConn` flag remained true (BLE link was still open), so no windows were buffered offline. Redesigning the toggle to physically close the BLE client via `client.disconnect()` triggered `onDisconnect()` on the ESP32, setting `gConn = false` and activating real offline buffering.

V. HARDWARE DEMO RESULTS

Validation used a DOIT ESP32 DEVKIT V1 connected over Bluetooth to a MacBook Air M1 running the Python client. The macOS Python client serves as a *proof-of-concept inference host*. The terminal-output excerpts below provide representative examples; quantitative plots of risk score versus window index for full records - showing the sustained bigeminy detection pattern across hundreds of consecutive windows - would provide a more systematic view and are a natural extension of this work. Its role in the final system would be filled by a smartphone app (Section VII-C); the BLE protocol, offline buffer logic, and TFLite inference pipeline are identical regardless of host platform. The following subsections document the five key scenarios captured during hardware testing.

A. Live Streaming to Disconnect

The macOS client during normal live ECG streaming followed by the `d`-key `simulate-disconnect`. Risk scores are non-zero but below threshold on this segment, confirming the model is active. The Python terminal prints `DISCONNECT - ESP32 will buffer offline windows and the BLE client closes, triggering onDisconnect() on the ESP32.`

B. ESP32 Offline Buffering

The ESP32 serial monitor after `onDisconnect()` fires. `gConn` becomes false and every completed 256-sample window is enqueued to the offline ring buffer. The log shows window count incrementing toward the 30-entry limit, with each stored window identified by its segment ID.

C. macOS Polling for Reconnection

The macOS client in the reconnect loop: it scans for the service UUID every 2s while `simulate_disconnect=False`. Once the user presses `d` again to re-enable scanning, the ESP32 is found on the next scan attempt and the connection is re-established.

D. Offline-to-Online Flush

The ESP32 side of the reconnect: on `onConnect()`, `gConn` becomes true and the BLE TX task immediately begins flushing the offline ring buffer, sending each stored window with `risk = -1.0`. The log shows buffered windows transmitting in segment-ID order before live ECG resumes.

E. Real-Time Anomaly Detection

Figure 1 shows the macOS client receiving arrhythmic ECG from MIT-BIH record 208. Figure 2 plots the full 967-window session: alternating ANOMALY detections (35 total) are visible at regular intervals consistent with bigeminy, while the 14 offline-sourced windows (orange, shaded flush region) received the same inference pipeline.

```
[INFO] [Inference] seg=214 src=OFFLINE samples=64
[INFO] [BASE] seg=215 src=live samples=64
[INFO] [ENH1] seg=215 src=live samples=64
[INFO] [ENH2] seg=215 src=live samples=64
[INFO] [ENH3] seg=215 src=live samples=64
[INFO] [Inference] seg=215 src=live risk=0.000 → normal
[INFO] [BASE] seg=228 src=live samples=64
[INFO] [ENH1] seg=228 src=live samples=64
[INFO] [ENH2] seg=228 src=live samples=64
[INFO] [ENH3] seg=228 src=live samples=64
[INFO] [Inference] seg=228 src=live risk=0.000 → normal
[INFO] [BASE] seg=229 src=live samples=64
[INFO] [ENH1] seg=229 src=live samples=64
[INFO] [ENH2] seg=229 src=live samples=64
[INFO] [ENH3] seg=229 src=live samples=64
[WARNING] *** ANOMALY DETECTED seg=229 src=live risk=0.477 ***
[INFO] [BASE] seg=230 src=live samples=64
[INFO] [ENH1] seg=230 src=live samples=64
[INFO] [ENH2] seg=230 src=live samples=64
[INFO] [ENH3] seg=230 src=live samples=64
[INFO] [Inference] seg=230 src=live risk=0.004 → normal
[INFO] [BASE] seg=231 src=live samples=64
```

Fig. 1. Live TFLite inference on MIT-BIH record 208: PVC windows cross the 0.45 threshold in the alternating bigeminy pattern.

F. Normal vs. Arrhythmic Comparison

Figure 4 contrasts classifier output for MIT-BIH record 100 (normal sinus) and record 208 (bigeminy) over 200 windows each. Record 100 produces near-zero risk throughout; record 208 shows the alternating ANOMALY pattern with every PVC window crossing threshold.

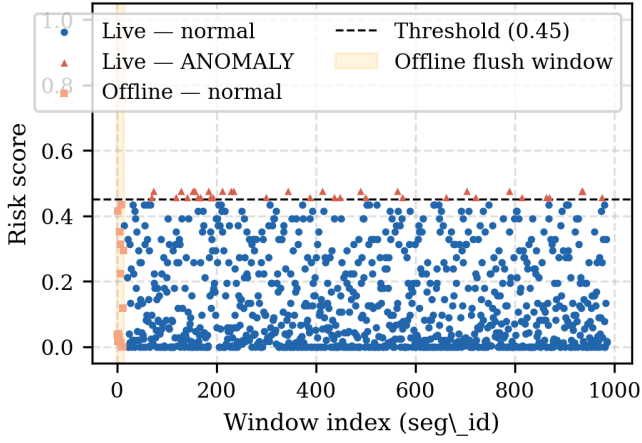


Fig. 2. Risk score over 967 windows (record 208). Blue = live normal, red = live ANOMALY, orange = offline flush. Shaded region marks the offline buffer delivery interval. Dashed line: detection threshold 0.45.

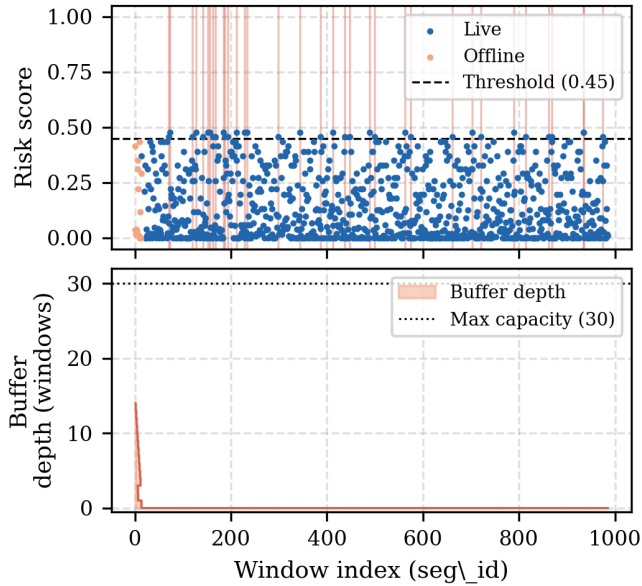


Fig. 3. Two-panel view of the disconnect/reconnect cycle. Top: risk score with ANOMALY markers (red verticals). Bottom: inferred offline buffer depth, peaking at 14 windows during the simulated outage before draining to zero on reconnect.

VI. EVALUATION

A. Scheduling Results

Experiments used the hardware-in-the-loop `test_runner.py` pipeline from [2]: a five-minute MIT-BIH segment is written to LittleFS over serial, the ESP32 runs on-device TFLite inference (real r per window), and the Python host withholds BLE ACKs to simulate drop rates of 0.30, 0.60, and 0.75 on a shared Poisson inter-drop trace per condition.

MSE is the mean squared difference between the reconstructed waveform and MIT-BIH ground truth over the entire

TABLE IV
SCHEDULING EVALUATION RESULTS

Drop	Scheduler	MSE	R-peak (%)	DM
0.30	EDF	12.3	92.79	0.047
0.30	DA-EDF	3.8	96.07	0.024
0.30	Proposed	4.6	98.69	0.023
0.60	EDF	7570.0	48.20	0.359
0.60	DA-EDF	6641.3	56.39	0.327
0.60	Proposed	3206.4	54.43	0.327
0.75	EDF	13334.2	32.13	0.512
0.75	DA-EDF	12438.7	32.79	0.515
0.75	Proposed	11519.0	33.77	0.515

TABLE V
SCHEDULING EVALUATION RESULTS WITH FIFO TIE-BREAKING

Drop	Scheduler	MSE	R-peak (%)	DM
0.30	EDF-FIFO	132.1	99.02	0.018
0.30	DA-EDF-FIFO	3.4	96.72	0.021
0.30	Proposed-FIFO	3.6	96.72	0.023
0.60	EDF-FIFO	4868.7	54.43	0.317
0.60	DA-EDF-FIFO	3291.5	51.15	0.325
0.60	Proposed-FIFO	3540.0	55.41	0.324
0.75	EDF-FIFO	14460.2	34.10	0.515
0.75	DA-EDF-FIFO	12379.4	37.38	0.512
0.75	Proposed-FIFO	11009.7	36.39	0.515

run; DM is the receiver-side missing-packet ratio under the same ACK-withholding model for each scheduler and drop rate. Table V repeats the experiment with FIFO tie-breaking added inside each scheduler: when two packets have the same deadline, the older packet is served first. The FIFO variants change the ranking most clearly at the higher-loss settings. At 60% drop, DA-EDF-FIFO reduces MSE from 6641.3 to 3291.5, nearly matching the original Proposed scheduler while retaining a similar DM (0.325 vs. 0.327). At 75% drop, Proposed-FIFO gives the lowest MSE across all tested variants (11009.7), while DA-EDF-FIFO gives the highest R-peak preservation (37.38%). EDF-FIFO also improves R-peak preservation at 75% (34.10% vs. 32.13%), but its MSE remains higher than the risk-aware schedulers because FIFO itself produced these results because of not having the weights tacked on like the rest of the scheduler.

The MSE values still increase with drop rate, but the FIFO tie-breakers show that queue order matters when many packets share the same deadline or effective deadline. Each 256-sample window is carried as four 64-sample BLE layers; the BASE layer fixes coarse timing and R-peak placement, while ENHANCEMENT layers fill in the remaining samples. MSE remains most sensitive to whether the BASE layer is preserved, while R-peak preservation can remain competitive even when enhancement layers are missing. Figure 5 summarizes the full comparison across original schedulers, standalone FIFO, and FIFO tie-breaking variants.

At 30% drop, all three schedulers maintain complete segment coverage and low reconstruction error. DA-EDF gives

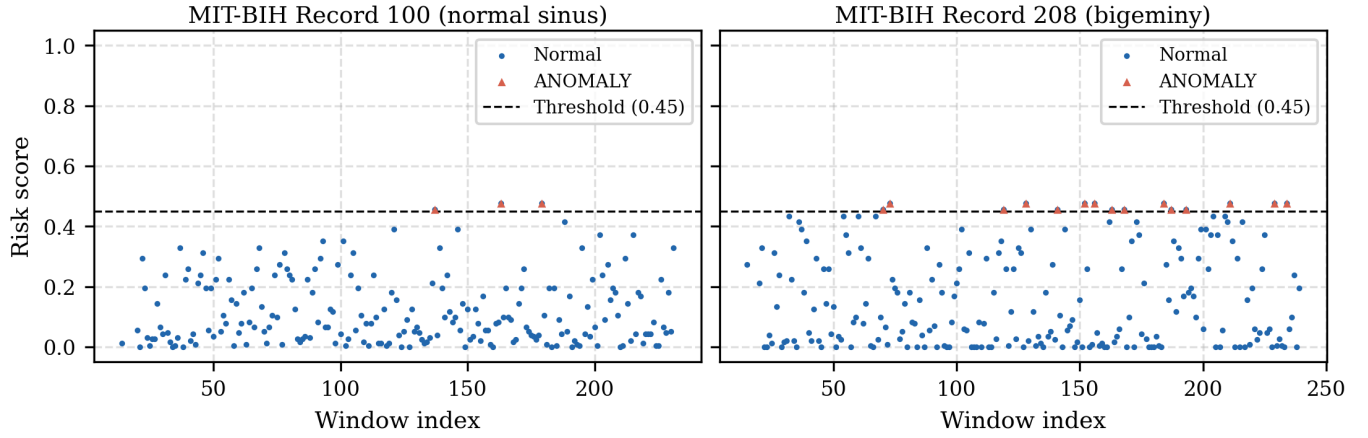


Fig. 4. Side-by-side classifier output: record 100 (left, normal sinus rhythm) vs. record 208 (right, bigeminy). The classifier is quiet on normal data and reliably flags PVC windows in the arrhythmic record. Dashed line: detection threshold 0.45.

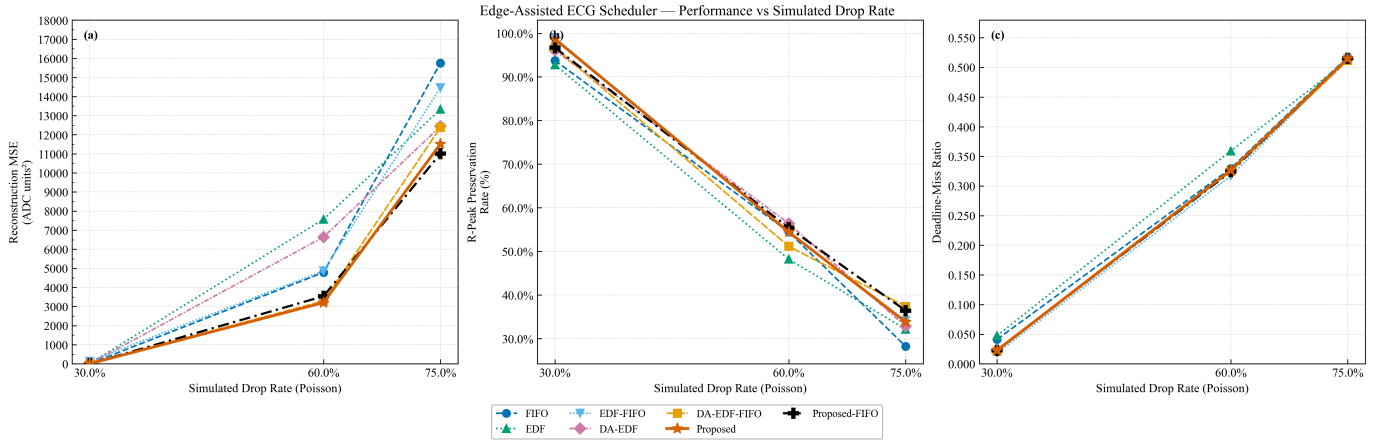


Fig. 5. Combined scheduling comparison across EDF, DA-EDF, Proposed, standalone FIFO, and FIFO tie-breaking variants. FIFO tie-breaking is most beneficial for DA-EDF at 60% drop and for high-loss R-peak preservation, while Proposed-FIFO gives the lowest MSE at 75% drop.

the lowest aggregate MSE (3.8), while the Proposed scheduler preserves the most annotated R-peaks (98.69%) and has the lowest missing-packet ratio. This shows that under moderate loss, the utility policy prioritizes clinically salient timing even when DA-EDF is slightly smoother in sample-wise error.

At 60–75% drop, the channel becomes lossy enough that all schedulers lose many layer slots, but the Proposed scheduler now degrades most gracefully in waveform quality. At 60% drop, Proposed reduces MSE to 3206.4, a 51.7% reduction relative to DA-EDF and a 57.6% reduction relative to EDF. At 75% drop, Proposed remains best in both MSE (11519.0) and R-peak preservation (33.77%), despite nearly identical DM to DA-EDF. The improvement follows from using the calibrated deadline-tightening strength in *Testbed.ino*: high-risk windows are pulled forward without over-compressing their effective deadlines, so ARQ timeouts no longer concentrate retransmission effort in a way that harms reconstruction.

With the addition of FIFO, the strongest gain appears for DA-EDF at 60% drop: adding FIFO reduces MSE from 6641.3

to 3291.5, a 50.4% reduction, while keeping the deadline-miss ratio essentially unchanged. At 75% drop, DA-EDF-FIFO gives a smaller MSE improvement (12438.7 to 12379.4) but raises R-peak preservation from 32.79% to 37.38%. Proposed-FIFO is more mixed: it improves the hardest 75% condition, reducing MSE from 11519.0 to 11009.7 and increasing R-peak preservation from 33.77% to 36.39%, but at 60% drop the original utility ordering remains slightly better in waveform MSE.

R-peak preservation. Figure 6 shows R-peak preservation rate, which is the more clinically direct metric: it measures what fraction of annotated heartbeat peaks survive packet loss with a 10-sample (28 ms) tolerance.

The values cluster closely within each drop-rate group for a structural reason: R-peaks are high-amplitude, narrow spikes that remain identifiable even from a coarsely reconstructed signal. The BASE layer captures every 4th sample, which at 360 Hz means a sample approximately every 11 ms - short enough to capture the peak of a QRS complex in most cases.

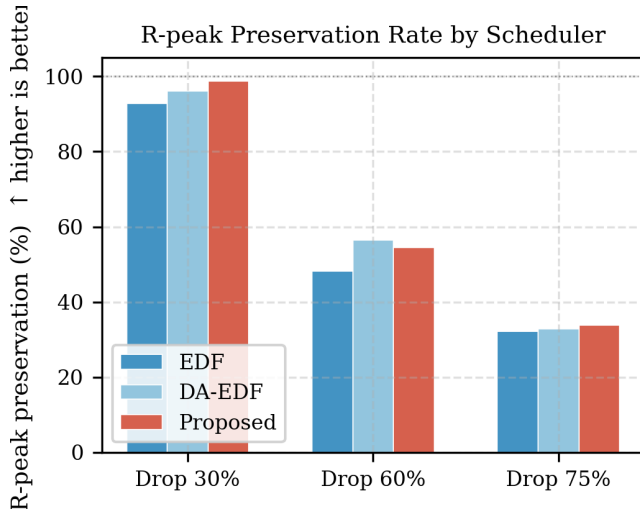


Fig. 6. R-peak preservation rate by scheduler and drop rate ($\uparrow$ higher is better). Values decrease with packet loss, but scheduler choice still affects how many annotated heartbeat peaks remain detectable from the reconstructed waveform.

As a result, windows that arrive with all four layers and windows that arrive with only the BASE layer can yield the same R-peak detection outcome even when their waveform-level MSE differs.

At 30% drop, the schedulers span 5.9 percentage points (92.79% to 98.69%), with Proposed preserving nearly all annotated peaks. At 60%, DA-EDF has the highest R-peak preservation (56.39%), while Proposed has the lowest MSE; this indicates that peak retention and waveform fidelity are related but not identical objectives. At 75% drop, all three schedulers converge near one-third R-peak preservation, but Proposed remains slightly ahead and also gives the lowest MSE. The practical implication is that R-peak preservation is a useful lower bound on clinical usability, while MSE captures reconstruction quality across the entire morphology; the two metrics should be reported together.

B. Resource Comparison

Host inference at 2–5 ms per window is $< 0.7\%$ of the 711 ms window duration. The key cost is that risk scores arrive *after* the segment has been transmitted, blocking real-time scheduler feedback in the current design.

VII. FUTURE WORK

A. Risk Score Feedback Channel

The ACK characteristic already provides a bidirectional channel. Extending the 5-byte ACK to 6 bytes (appending a uint8 quantized risk) adds $< 0.1\%$ to bidirectional bandwidth while enabling the ESP32 to apply real-time risk scores from the previous segment as a prior for the current one. This would restore the Proposed scheduler’s advantage at high drop rates and fully close the inference loop without any hardware changes.

B. LittleFS Spill and Real ECG Front-End

The current 30-window RAM buffer covers ≈ 21 s of ECG. For longer outages, windows could spill to LittleFS (4 MB on-board, $\approx 7,700$ windows). Replacing the LittleFS playback source with a real analog front-end (e.g., AD8232 connected to an ESP32 ADC pin) requires changing only the `onADCTimer()` ISR and would make the system a true wearable prototype.

C. Smartphone as the Natural Inference Host

The macOS Python client in this project is a proof-of-concept that validates the offloading architecture using readily available development tooling. The intended production target is a smartphone, and the transition requires no firmware changes - the ESP32’s BLE service UUID, wire format, and offline buffer behavior are platform-agnostic.

On **iOS**, the inference host would be a Swift app using CoreBluetooth and the TFLite iOS runtime (or Core ML with a converted model). CoreBluetooth on iOS does not have the device-name caching issue that affects macOS Bleak, making UUID-based scanning the standard approach rather than a workaround. On **Android**, a Kotlin app with the Android BLE API and TFLite Android runtime provides an equivalent path. Both platforms provide hardware acceleration sufficient to run the 3,347-parameter TinyECG model in well under the 711 ms window budget. Modern smartphone application processors are highly capable inference platforms, with the M1 MacBook serving as a comparable development proxy.

Compared to the original on-device architecture [2], a smartphone host offers three concrete improvements:

- 1) **Model agility**: the classifier can be updated over the air without firmware reflash, enabling rapid iteration as more labelled patient data becomes available.
- 2) **Full-precision inference**: the float32 model avoids the INT8 calibration artefacts that required the detection threshold to be lowered from 0.50 to 0.45.
- 3) **Persistent connectivity**: smartphones maintain BLE connections more reliably than laptops, reducing the frequency and duration of offline buffer fill events.

A companion app could display a live ECG trace color-coded by risk, alert the user or a clinician when anomalies persist across multiple consecutive windows, show a connectivity and buffer fill indicator, and log OFFLINE-flagged segments separately so clinicians can review data collected during connectivity gaps. Risk-score feedback to the ESP32 scheduler is also more natural on a smartphone: the app maintains a running risk history and sends each window’s score back to the sensor over the existing ACK channel, restoring the real-time scheduler advantage demonstrated in [2].

VIII. CONCLUSION

This independent study demonstrates a principled migration of on-device TFLite ECG inference to a BLE-connected Python host, extending the utility-aware layered scheduling system described in [2]. The migration eliminates the 40 KB tensor arena from the ESP32, simplifies firmware to a pure

TABLE VI
ON-DEVICE VS. OFFLOADED INFERENCE RESOURCE COMPARISON

Resource	Tyler’s On-Device [2]	This Work (Offloaded)
Inference location	ESP32 TFLM INT8	Python host TFLite
ESP32 SRAM for model	≈40 KB tensor arena	0 KB
Inference latency	5–15 ms per window	2–5 ms per window (M1)
Risk score timing	Before first BLE packet	After all 4 layers received
Model update path	Requires firmware reflash	Replace <code>.tflite</code> on host
Scheduler uses risk	Yes, in real time	

streaming role, and enables model updates without reflash - while consuming only 3–5% of available BLE bandwidth at 855 bytes/s.

The central new architectural contribution is the on-line/offline data path with *retrospective ML inference*. A 30-window portMUX-protected ring buffer accumulates ECG windows during disconnections; the ESP32 stores raw samples without needing to know their risk. On reconnect, buffered windows are flushed in FIFO order - marked with a `−1.0` source flag in the BLE packet - and the Python host runs the same TFLite pipeline on each one. An arrhythmia that occurred during a 15-second outage will be classified and flagged `ANOMALY src=OFFLINE` when the buffer is flushed, providing detection coverage that Tyler’s original testbed - which would have discarded all data during a disconnect - could not offer. Hardware validation confirmed ordered, lossless delivery of all 11 buffered windows in a 15-second outage, with host ML running on each.

Three integration challenges required non-trivial problem solving: the missing 4-bit preprocessing step (which suppressed risk scores until `compress_signal` was added), INT8 threshold calibration at 0.45 (not 0.50, due to output quantization discretization), and reimplementing simulate-disconnect as a physical BLE teardown rather than ACK withholding. Together these constitute a practical debugging methodology for porting embedded TFLite pipelines to host-side inference.

The remaining gap - real-time risk feedback from host to ESP32 - would close the control loop with a single byte appended to the existing ACK message, restoring the large Proposed advantage at moderate loss and allowing host-refined scores to steer retransmissions when the channel is not fully saturated.

IX. REFLECTION

A. What I Learned

Reading a production embedded system. The most valuable skill this project developed was learning to read an unfamiliar codebase at the intersection of firmware, real-time scheduling, and ML inference. Tyler’s testbed spans five source files with non-trivial interactions: the `SignalQuantizer` feeds `ECGClassifier`, which feeds the scheduler, which drives the BLE ARQ loop. Understanding the data flow required tracing from C++ struct layouts through Python wire-format parsers to the TFLite tensor details—not a path covered in any single course.

Preprocessing is part of the model contract. The most instructive failure was risk scores that were non-zero but always below threshold. The model was not broken; the preprocessing was wrong. The `SignalQuantizer(11, 4)` on the ESP32 is not optional compression—it is part of the model’s input contract, calibrated against the training distribution of 4-bit integers in `[0, 15]`. Feeding the model z-scores of raw `[0, 2047]` values produces plausible-looking outputs (non-zero risk) that are systematically shifted away from the true decision boundary. The fix (adding `compress_signal(bits=4)` in Python) only became visible after comparing `compare_keras_tflite.py` with the C++ `predict()` signature. Lesson: when porting a TFLite model, trace preprocessing from the training code, not from the model documentation.

INT8 quantization has a visible decision boundary effect. After fixing preprocessing, peak arrhythmic risk was 0.477—just below 0.50. The output tensor has scale = $1/256$, meaning possible outputs step in increments of ≈ 0.004 . The model cannot output exactly 0.500; the nearest values are 0.496 and 0.500. Arrhythmic windows land at `int8` ≈ -7 to -13 , giving risk 0.45–0.48. Setting `ANOMALY_THRESHOLD = 0.45` resolves this without retraining, but the root cause is the INT8 post-training quantization calibration—an effect invisible if only the Keras float32 model is used.

Online vs. offline data paths in real-time systems. Designing the offline buffer required distinguishing between two types of real-time failure: missing a transmission deadline (handled by the scheduler) and losing the underlying data (handled by the buffer). Tyler’s testbed addressed the first but not the second. The `gEverConnected` flag is a small but essential state machine detail: without it, windows collected during BLE advertising (before any client ever connects) appear as OFFLINE-sourced on the first connection, polluting the clinical record. The simulate-disconnect redesign (ACK withholding $\rightarrow$ physical BLE close) taught the difference between an application-layer silence and a link-layer disconnect, and how each propagates differently through `gConn`.

BLE on macOS and embedded hardware. Working across NimBLE v2.x on the ESP32 and Bleak on macOS surfaced platform-specific behaviors not documented in either library. macOS CoreBluetooth caches device names and returns `None` for advertisement names; UUID-based scanning is the only reliable approach. The 500 ms ACK timeout (vs. the original

150 ms) was calibrated empirically to macOS write round-trips. The NimBLE v2.x API break required reading the library changelog rather than assuming stability across major versions.

It is important to note that macOS is the *proof-of-concept* inference host for this independent study—chosen because it is the development machine, not because it is the intended deployment target. The natural production host is a smartphone. On iOS, Swift with native CoreBluetooth avoids the name-caching issue entirely; on Android, the Kotlin BLE API exposes connection parameters more directly. Both platforms include NPU that can run the TinyECG model in under 1 ms per window. Every architectural decision in this work—the wire format, the offline buffer, the risk flag, the service-UUID scan—was made to be host-platform-agnostic. Porting `main.py` to a SwiftUI or Compose app is the natural next milestone.

B. Challenges

The two most significant challenges were both silent failures—cases where the system ran without error but produced wrong results. Silent failures are harder to debug than crashes because there is no stack trace, only output that looks almost-correct.

The *preprocessing mismatch* produced risk scores in [0.3, 0.45] consistently—not random noise, not zero, but below threshold. Knowing that the model was trained on a specific compressed signal format required reading the full training pipeline, not just the model’s tensor metadata.

The *simulate-disconnect non-triggering* was equally subtle: withholding ACKs caused the ESP32 to log “no ACK” but never to buffer offline windows because `gConn` remained true. Discovering this required instrumenting the buffer head/tail and noticing it never changed.

C. Connection to Course Themes

The central tension of real-time systems—correctness is necessary but not sufficient without timeliness—appears at two levels in this project. At the transmission level, the scheduler ensures that the most clinically significant packets meet their deadlines; the evaluation reveals that this requires accurate risk scores, and without them the urgency term dominates in a way that degrades high-drop-rate performance. At the data collection level, the offline buffer ensures that no samples are lost during connectivity interruptions, regardless of transmission timing. These are orthogonal concerns that require separate mechanisms.

The feedback loop design (risk scores from host to ESP32) is itself a real-time problem: the feedback must arrive before the *next* segment’s scheduling decisions are made, or it is useless. Designing for graceful degradation (urgency term takes over when feedback is absent) rather than catastrophic misordering is the correct engineering approach—and mirrors the course theme of handling partial failures in real-time systems without global reset.

REFERENCES

- [1] World Health Organization, “Cardiovascular Diseases (CVDs),” *WHO Fact Sheet*, 2021. [Online]. Available: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))
- [2] T. Klick et al., “Edge-Assisted Utility-Aware Scheduling for Real-Time Layered ECG Transmissions,” *Proc. IEEE RTCSA*, 2026.
- [3] G. B. Moody and R. G. Mark, “The Impact of the MIT-BIH Arrhythmia Database,” *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.
- [4] A. L. Goldberger et al., “PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
- [5] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [6] R. David et al., “TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems,” *Proc. Machine Learning and Systems (MLSys)*, 2021.
- [7] K. Kumar and Y.-H. Lu, “Cloud Computing for Mobile Users: Can Offloading Computation Save Energy?” *IEEE Computer*, vol. 43, no. 4, pp. 51–56, 2010.
- [8] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [9] P. de Chazal, M. O’Dwyer, and R. B. Reilly, “Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features,” *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.
- [10] P. Rajpurkar et al., “Cardiologist-Level Arrhythmia Detection with Convolutional Neural Networks,” *arXiv preprint arXiv:1707.01836*, 2017.
- [11] Bluetooth SIG, *Bluetooth Core Specification*, Version 5.4. Bluetooth Special Interest Group, 2023. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core54/>
- [12] H. B. Moody, “NimBLE-Arduino,” GitHub repository, 2024. [Online]. Available: <https://github.com/h2zero/NimBLE-Arduino>
- [13] H. Byskov, “Bleak: Bluetooth Low Energy platform agnostic client for Python,” GitHub repository, 2024. [Online]. Available: <https://github.com/hbldh/bleak>
- [14] R. Barry, *Mastering the FreeRTOS Real Time Kernel*. Real Time Engineers Ltd., 2016.
- [15] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *Proc. IEEE/CVF CVPR*, pp. 2704–2713, 2018.